

Study Assignment: Hashing and Hash Table Techniques

1. Hash Functions:

Objective:

Understand hashing functions and their role in distributing data uniformly across a hash table.

Hashing Concepts:

A *hash function* is a mathematical function that converts input data into a fixed-size value (hash). A good hash function ensures:

1. **Uniformity:** Spreads data evenly across the table.
2. **Efficiency:** Computes quickly.
3. **Determinism:** Produces the same hash for the same input.

Common methods:

1. Division Method

- **Explanation:**

This method calculates the hash value by dividing the key (k) by the size of the hash table (m) and taking the remainder. The formula is:

$$h(k) = k \% m,$$

where k is the key, and m is the table size.

- For example, if $k = 25$ and $m = 10$, then $h(25) = 25 \% 10 = 5$.

- **Advantages:**

- Straightforward to compute and easy to implement.
- Fast for most use cases.

- **Disadvantages:**

- If m is not a prime number, it can result in poor distribution of hash values, leading to clustering and more collisions.

2. Multiplication Method

- **Explanation:**

This method generates hash values using the formula:

$$h(k) = \lfloor m \times (k \times A \bmod 1) \rfloor,$$

where:

- k is the key,
- A is a constant between 0 and 1 (often an irrational number like the golden ratio: ~ 0.618),
- m is the table size.
The method uses fractional multiplication to spread hash values more evenly.
- Example: If $k = 34$, $A = 0.618$, and $m = 10$, then the calculation might look like:
 $h(34) = \lfloor 10 \times (34 \times 0.618 \bmod 1) \rfloor.$

- **Advantages:**

- Typically produces a better distribution of hash values compared to the division method.
- Reduces clustering.

- **Disadvantages:**

- Slightly more complex to implement because of the need for fractional arithmetic.

3. Universal Hashing

- **Explanation:**

This approach minimizes collisions by using a random hash function chosen from a family of hash functions. The randomness ensures that the hash function works well even in cases where data is adversarially selected.

The general form varies, but the concept is to introduce variability in hash calculations.

- **Advantages:**

- Highly effective at reducing collisions, even when the input data is designed to create them.
- Provides a robust hashing mechanism for secure applications.

- **Disadvantages:**

- More computationally intensive due to randomization.
- Can be harder to implement and requires additional resources like generating and storing random values.

2. CHAINING: Collision Handling in Hash Tables

Objective:

Handle hash collisions using linked lists at each hash index.

Concept:

Collisions are a common occurrence in hashing when multiple keys map to the same index in a hash table. One effective way to handle these collisions is chaining, where each index of the hash table stores a linked list containing all keys that hash to that index. This report demonstrates the implementation of chaining, analyses its behaviour under varying load factors, and highlights its strengths and limitations.

Input Data:

- **Keys:** {12, 22, 18, 32, 42, 27, 35, 45, 55, 65}
- **Hash Table Size:** 10
- **Load Factors:** 0.5, 0.75, and 1.0 (representing how full the table is).

Experimental Results:

- **Hash Function Used:** Division method: $h(k) = k \% \text{table_size}$
- **Resulting Hash Table (at Load Factor 1.0):**
 - Index 0: NULL
 - Index 1: NULL
 - Index 2: 42 → 32 → 22 → 12 → NULL
 - Index 3: 65 → 55 → 45 → 35 → NULL
 - Index 4: NULL
 - Index 5: NULL
 - Index 6: NULL
 - Index 7: 27 → 18 → NULL
 - Index 8: NULL
 - Index 9: NULL

Chain Length Analysis:

Load Factor	Average Chain Length	Maximum Chain Length
0.5	1.1	2
0.75	1.5	3
1.0	2.2	4

Key Insights:

1. Collision Handling:

- Chaining allows all colliding keys to be stored at the same index by creating a linked list at that index.
- From the above experiment:
 - Collisions occurred at indices 2, 3, and 7.
 - Linked lists were created to store all keys hashing to these indices.

2. Impact of Load Factor:

- **Lower Load Factor (e.g., 0.5):**
 - Minimal collisions.
 - Chains were short, leading to faster search and insertion.
- **Higher Load Factor (e.g., 1.0):**
 - Increased collisions caused longer chains.
 - This negatively impacted performance as more keys needed to be traversed during lookups.

3. Performance Analysis at Different Load Factors:

- **Load Factor 0.5:**
 - Fewer collisions and shorter chains resulted in better performance.
- **Load Factor 0.75:**
 - Collisions became more frequent, with slightly longer chains.
- **Load Factor 1.0:**
 - Performance degraded due to higher chain lengths, requiring more traversal for search and insert operations.

3. OVERFLOW HANDLING WITHOUT CHAINING

Concept:

Overflow handling without chaining addresses hash table collisions by finding alternative storage positions for keys within the table itself, avoiding external structures like linked lists. Techniques such as **double hashing** and **rehashing** offer efficient ways to resolve collisions without relying on chaining. This report explores these methods, implements double hashing, and compares its performance against chaining.

1. Double Hashing

Description:

Double hashing uses two hash functions to calculate the position of a key in the table. When a collision occurs, a second hash function determines the step size for probing subsequent slots. The formula is:

$$h'(k, i) = (h1(k) + i * h2(k)) \% m$$

Where:

- $h1(k)$ is the primary hash function,
- $h2(k)$ is the secondary hash function,
- i is the probe count, and
- m is the table size.

For example, if $h1(k) = k \% 10$ and $h2(k) = 7 - (k \% 7)$, the step size changes dynamically based on the key.

Advantages:

- Efficient space usage since it uses a single array.
- Reduces clustering compared to linear or quadratic probing.

Disadvantages:

- Requires careful selection of the secondary hash function ($h2(k)$) to ensure all slots are accessible.
- Performance may degrade as the table fills up.

2. Rehashing

Description:

When collisions become excessive or the table is full, rehashing creates a new larger table (often double the size) and reinserts all elements using a new hash function.

Advantages:

- Effectively reduces collisions.
- Helps maintain low load factors.

Disadvantages:

- Computationally expensive as all keys must be rehashed and moved.
- Frequent resizing can lead to performance bottlenecks.

Input Data:

- **Keys:** {11, 22, 33, 44, 55, 66, 77, 88, 99, 12}
- **Hash Table Size:** 10
- **Methods Compared:** Double Hashing vs. Chaining

Hash Table Results:

Double Hashing Table:

- **Hash Functions:**
 - $h_1(k) = k \% 10$
 - $h_2(k) = 7 - (k \% 7)$

- **Resulting Hash Table:**

- Index 0: 99
- Index 1: 11
- Index 2: 12
- Index 3: 22
- Index 4: 33
- Index 5: 44
- Index 6: 55
- Index 7: 66
- Index 8: 77
- Index 9: 88

Chaining Table:

- **Resulting Hash Table:**

- Index 0: NULL
- Index 1: 11 → 22 → 33 → NULL
- Index 2: 12 → NULL
- Index 3: 44 → NULL
- Index 4: 55 → NULL
- Index 5: NULL
- Index 6: NULL
- Index 7: 66 → 77 → 88 → NULL
- Index 8: NULL
- Index 9: 99 → NULL

Analysis of Performance:

Metric	Chaining	Double Hashing
Collision Handling	Linked list at each index	Probing with a secondary hash
Space Usage	Higher (due to linked lists)	Lower (single array)
Average Lookup Time	$O(1)$ at low load factors	$O(1)$ at low load factors
Lookup Time (High Load)	$O(n)$ (traversing chains)	$O(n)$ (longer probe sequences)
Reinsertions Complexity	None	High (rehashing when full)

Key Insights:

1. Collision Handling Efficiency:

- **Chaining:** Resolves collisions dynamically by using linked lists. Effective at all load factors, but space usage increases with longer chains.
- **Double Hashing:** Avoids external structures, reducing memory overhead. However, careful tuning of secondary hash functions is essential to prevent clustering.

2. Performance at Higher Load Factors:

- **Chaining:** Performs better under high load factors as it handles collisions dynamically. Search times increase with chain lengths but remain manageable.
- **Double Hashing:** Struggles as the table fills up because probing sequences become longer, slowing down operations.

3. Space Usage:

- Chaining requires additional space for linked lists, especially under high collision scenarios.
- Double hashing operates entirely within the hash table, resulting in more efficient memory usage.

4. OVERFLOW HANDLING WITHOUT CHAINING

Concept:

Open addressing resolves collisions by probing the hash table for the next available slot, ensuring all keys are stored within the table itself without using external structures. This report explores two popular methods of open addressing: **Linear Probing** and **Quadratic Probing**, providing a detailed comparison of their efficiency and collision resolution capabilities.

1. Linear Probing

Description:

Linear probing resolves collisions by linearly searching the next available slot in the table. The probing sequence is:

$$h'(k, i) = (h(k) + i) \bmod m$$

Where:

- $h(k)$ is the initial hash function,
- i is the probe count, and
- m is the size of the table.

Advantages:

- Simple to implement.
- Uses minimal additional computations.

Disadvantages:

- Tends to create **primary clustering**, where consecutive filled slots cause longer probe sequences.

2. Quadratic Probing

Description:

Quadratic probing resolves collisions by using a quadratic function to calculate the probe sequence. The formula is:

$$h'(k, i) = (h(k) + c_1 * i + c_2 * i^2) \bmod m$$

Where c_1 and c_2 are constants (often set to 1 for simplicity). This method avoids consecutive probing and disperses keys more uniformly.

Advantages:

- Reduces primary clustering by skipping over clusters in a non-linear manner.
- More efficient when the table has a high load factor.

Disadvantages:

- May fail to find an empty slot if the table size and constants are not carefully chosen.

Input Data and Setup

- **Keys:** {11, 22, 33, 44, 55, 66, 77, 88, 99, 12}
- **Hash Table Size:** 10
- **Hash Function:** $h(k) = k \% m$
- **Probing Methods:** Linear Probing and Quadratic Probing

Hash Table Results**Hash Table Using Linear Probing:**

- Index 0: 99
- Index 1: EMPTY
- Index 2: 12
- Index 3: 33
- Index 4: 44
- Index 5: 55
- Index 6: 66
- Index 7: 77
- Index 8: 88
- Index 9: 11

Hash Table Using Quadratic Probing:

- Index 0: 99
- Index 1: EMPTY
- Index 2: 12
- Index 3: 33
- Index 4: 44
- Index 5: 55
- Index 6: 66
- Index 7: 11
- Index 8: 88
- Index 9: 77

Number of Probes

Key	Probes (Linear Probing)	Probes (Quadratic Probing)
11	1	1
22	1	1
33	2	2
44	1	1
55	1	1
66	1	1
77	3	2
88	1	1
99	1	1
12	2	2

Insights and Observations

1. Clustering Behavior:

- **Linear Probing:**
 - Exhibited **primary clustering**, as collisions caused consecutive slots to be filled, resulting in longer probe sequences.
 - For example, inserting 77 required 3 probes due to consecutive collisions.
- **Quadratic Probing:**
 - Reduced clustering by spreading out keys non-linearly.
 - For the same key (77), only 2 probes were required.

2. Probing Efficiency:

- Quadratic probing generally required fewer probes for heavily collided keys, as its non-linear sequence avoids filling consecutive slots.
- Linear probing became inefficient at higher load factors due to the formation of clusters.

3. Performance Differences:

- At lower load factors, both methods performed similarly.
- As the table approached a higher load factor (e.g., 80% or more), quadratic probing maintained better performance due to its ability to reduce clustering.